

Program 3: Code a Dockerized Python Flask or [Node.js](#) Application

Folder Structure

DEVOPSLAB

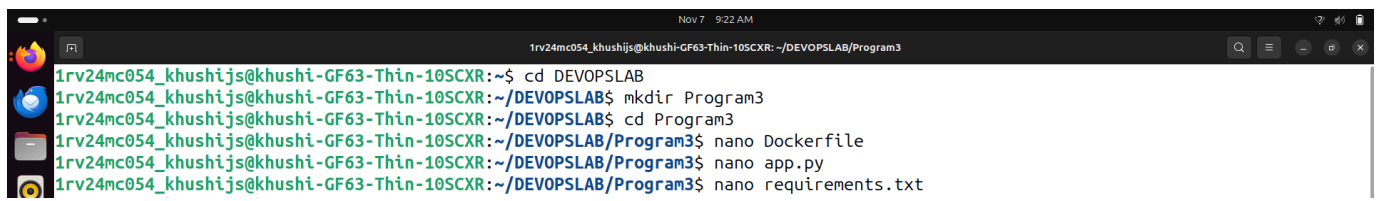
|--Program2

|--Dockerfile

|--app.py

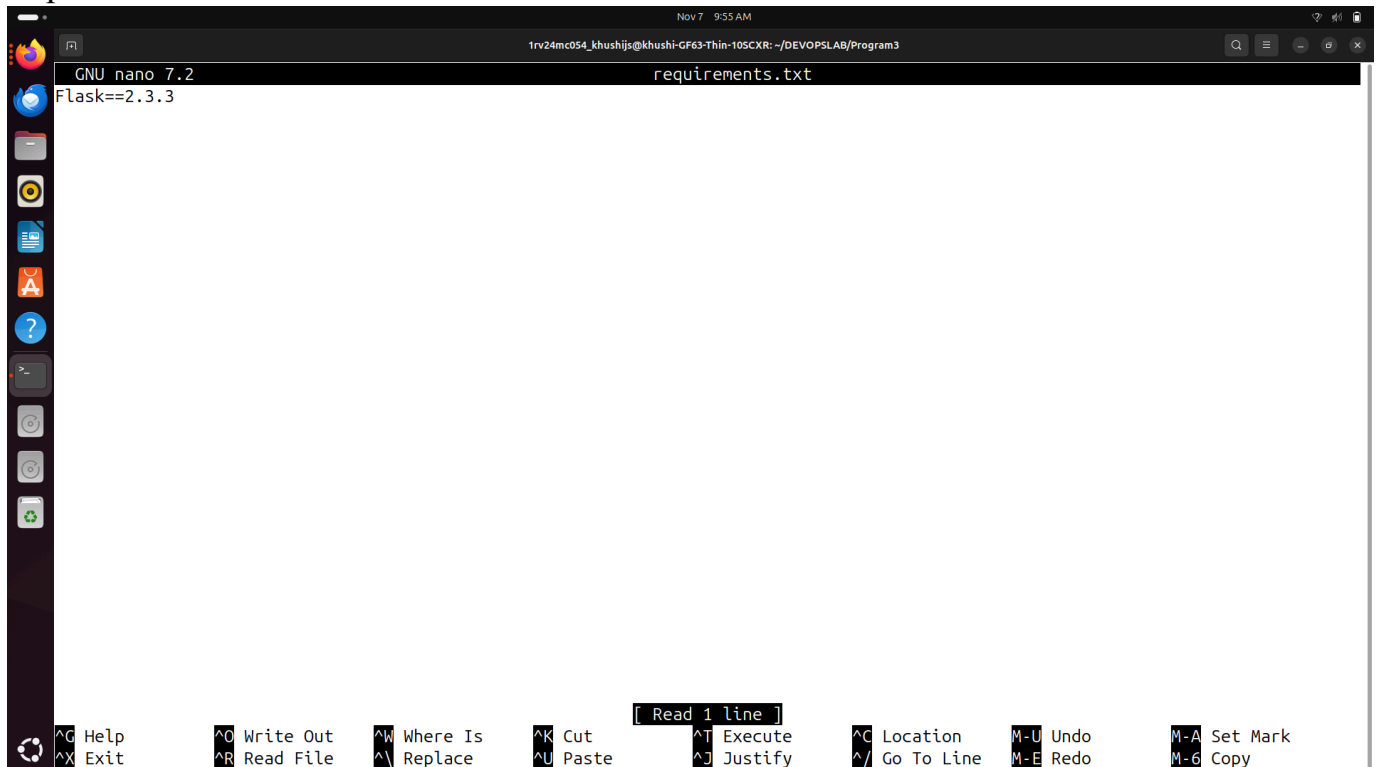
|--requirements.txt

Step 1: I've created a directory called DEVOPSLAB navigate to DEVOPSLAB directory and within that create a sub-directory with name Program3, navigate to Program3 directory and create the files such as app.py, requirements.txt and Dockerfile.



```
1rv24mc054_khushijs@khushi-GF63-Thin-10SCXR: ~$ cd DEVOPSLAB
1rv24mc054_khushijs@khushi-GF63-Thin-10SCXR: ~/DEVOPSLAB$ mkdir Program3
1rv24mc054_khushijs@khushi-GF63-Thin-10SCXR: ~/DEVOPSLAB$ cd Program3
1rv24mc054_khushijs@khushi-GF63-Thin-10SCXR: ~/DEVOPSLAB/Program3$ nano Dockerfile
1rv24mc054_khushijs@khushi-GF63-Thin-10SCXR: ~/DEVOPSLAB/Program3$ nano app.py
1rv24mc054_khushijs@khushi-GF63-Thin-10SCXR: ~/DEVOPSLAB/Program3$ nano requirements.txt
```

Step 2: Add the following code in requirements.txt file, which is used to install the Dependencies.

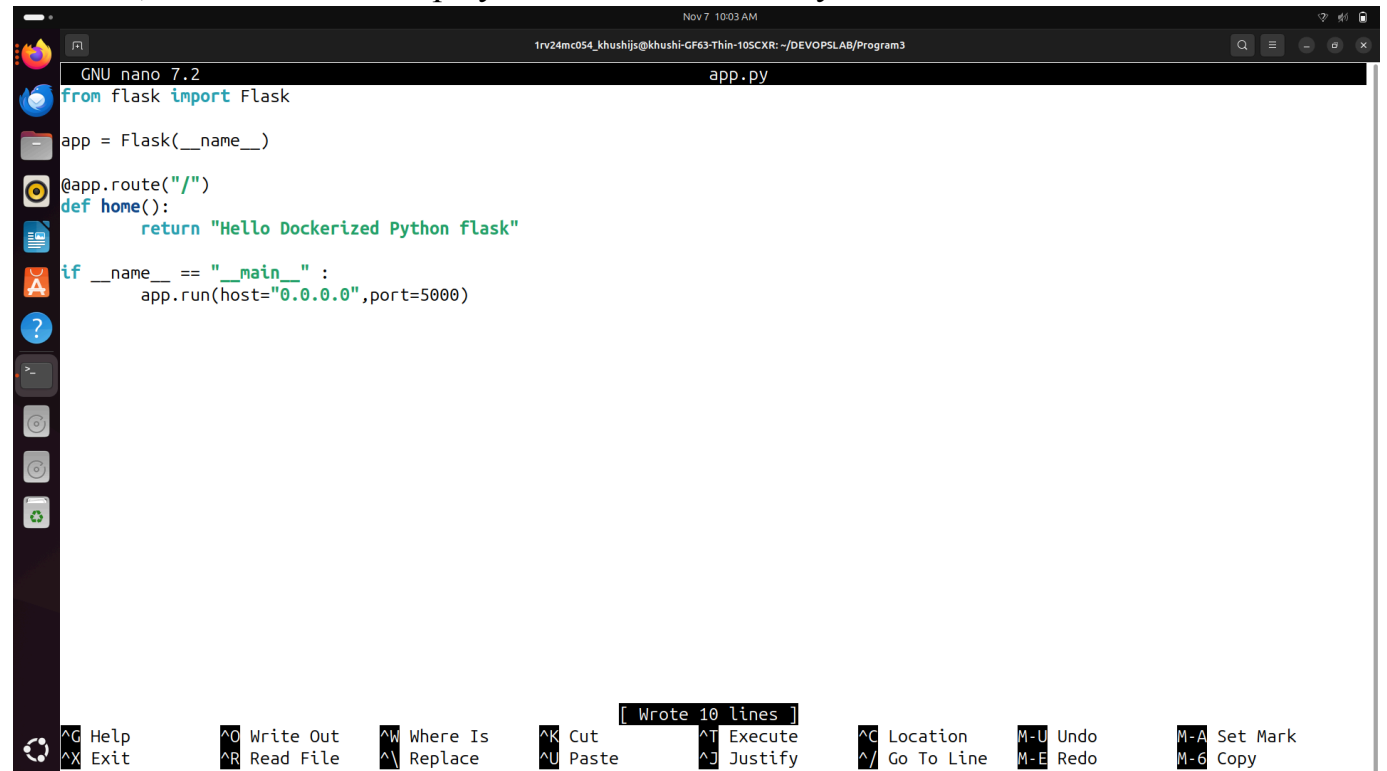


```
GNU nano 7.2 requirements.txt
Flask==2.3.3
```

[Read 1 line]

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Location	M-U Undo	M-A Set Mark
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo	M-6 Copy

Step 3: In the app.py file, implement the following code to run a basic web server. When accessed, this server will display "Hello Dockerized Python flask".



```
GNU nano 7.2 app.py
from flask import Flask

app = Flask(__name__)

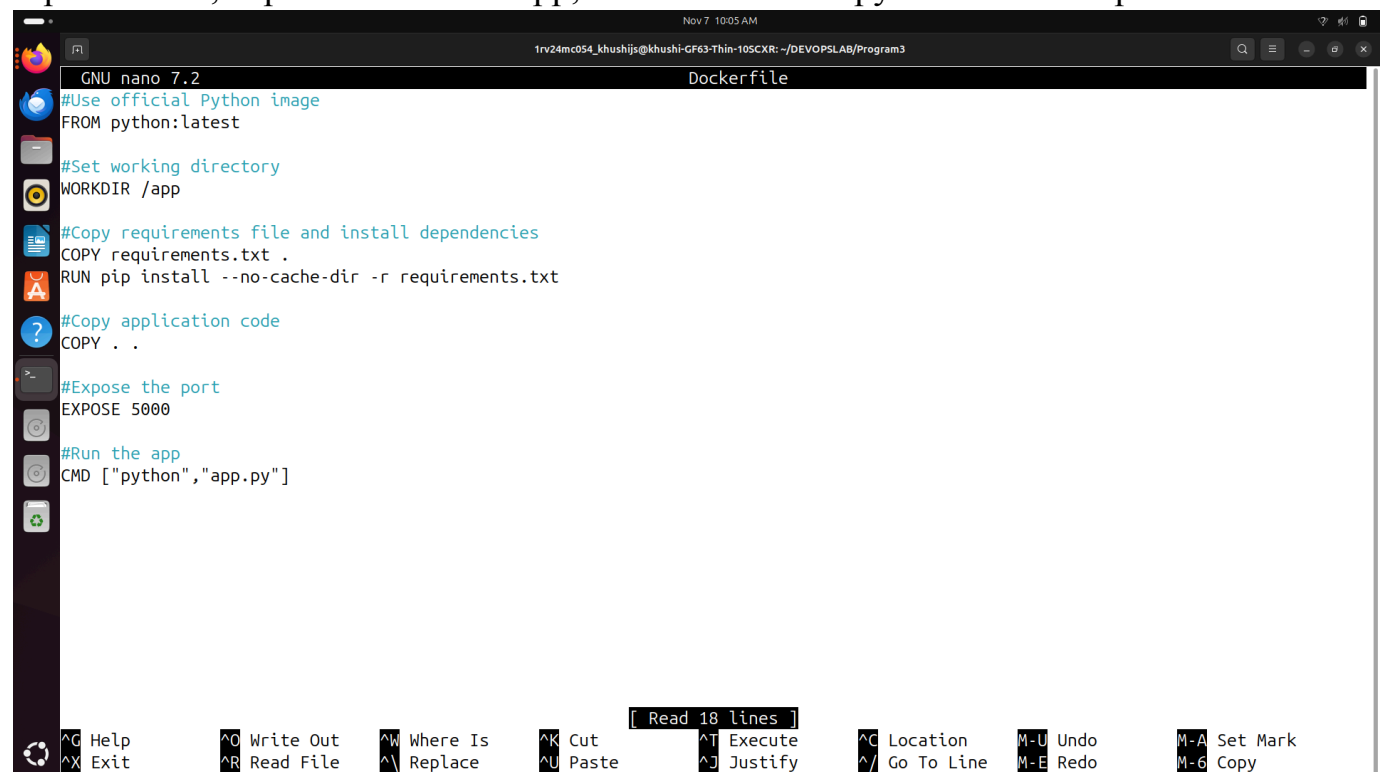
@app.route("/")
def home():
    return "Hello Dockerized Python flask"

if __name__ == "__main__":
    app.run(host="0.0.0.0",port=5000)
```

[Wrote 10 lines]

Help Write Out Where Is Cut Execute Location M-U Undo M-A Set Mark
Exit Read File Replace Paste Justify Go To Line M-E Redo M-6 Copy

Step 4: The following Dockerfile creates a lightweight container. It installs Python dependencies, copies the code to /app, and then runs the python server on port 5000.



```
GNU nano 7.2 Dockerfile
#Use official Python image
FROM python:latest

#Set working directory
WORKDIR /app

#Copy requirements file and install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

#Copy application code
COPY . .

#Expose the port
EXPOSE 5000

#Run the app
CMD ["python", "app.py"]
```

[Read 18 lines]

Help Write Out Where Is Cut Execute Location M-U Undo M-A Set Mark
Exit Read File Replace Paste Justify Go To Line M-E Redo M-6 Copy

Step 5: Next, use the docker build command to create a new image from the Dockerfile.

```
1rv24mc054_khushijs@khushi-GF63-Thin-10SCXR:~/DEVOPSLAB/Program3$ docker build -t program-3 .
[+] Building 6.2s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 349B
=> [internal] load metadata for docker.io/library/python:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:latest@sha256:1ad1a43b5e2478e62056bbc28028afd858185d73bf4d6a439cbb058b6800a96d
=> [internal] load build context
=> => transferring context: 624B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt .
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => writing image sha256:5d0598deb9e670ee162d1a436dbd99715e8a3a3964b560a9cc0014cf3c30d9c8
=> => naming to docker.io/library/program-3
```

Step 6: Execute the docker run command to create a container from the previously built Image.

```
1rv24mc054_khushijs@khushi-GF63-Thin-10SCXR:~/DEVOPSLAB/Program3$ docker run -p 5000:5000 program-3
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.18.0.2:5000
Press CTRL+C to quit
172.18.0.1 - - [07/Nov/2025 04:05:17] "GET / HTTP/1.1" 200 -
```

